

CST1450 Coursework Report

This coursework implements a graph program to find the shortest path between two vertices using Dijkstra's algorithm. The program includes an Edge class, a Graph class, a function to build a graph from a list of triples, and a function to load graph data from a JSON file. The graph is treated as undirected, so an edge from A to B is the same as an edge from B to A.

To use the program, place all .py and .json files in the same folder. Run the program from the command line using Python with:

```
python main.py
```

The program will then ask for the JSON filename. It automatically finds the shortest path from vertex A to vertex Z. No external libraries are required because the program only uses the standard Python library, including json.

The results for the provided graphs were as follows. For graph_1.json, the shortest path from A to Z was A -> B -> C -> Z with total distance 4.0. For graph_2.json, the graph was processed successfully and produced the same shortest path A -> B -> C -> Z with total distance 4.0. For graph_3.json, the program raised an exception because vertex Z is not present in the graph. For graph_4.json, the shortest path from A to Z was A -> B -> E -> I -> F -> G -> L -> R -> P -> T -> V -> X -> Y -> Z with total distance 18.0. For graph_5.json, the program raised an exception because no path exists between A and Z.

The custom graph, my_graph.json, contains 8 vertices: A, B, C, D, E, F, G and Z, and includes a valid path from A to Z. For this graph, the shortest path was A -> B -> D -> F -> G -> Z with total distance 13.0. I checked this by comparing it with other possible routes. For example, A -> B -> D -> Z has distance 15, A -> C -> D -> F -> G -> Z has distance 15, and A -> C -> E -> G -> Z has distance 16. Since 13.0 is the smallest total, the path returned by the program is the shortest.

The program raises an exception if an edge connects a vertex to itself, because this is not a valid road in this problem. It also raises an exception if a vertex is not present in the graph or if no path exists between two vertices.

One possible improvement would be to add a visual representation of the graph and highlight the shortest path found. This would make the output easier to understand.